ResumeMatch: AI-Powered Resume–Job Matching for Job Seekers

Zhuwei (Vivian) Xu
NYU Text as Data, Spring 2026
Instructor: Prof. Elliott Ash

Project URL: https://resumematch.pages.dev
GitHub: https://github.com/Vivian-ZW-Xu/ResumeMatch

TL;DR. ResumeMatch is a cloud-deployed (Render + Cloudflare Pages) resume–JD matching system that generates a custom evaluation rubric per JD, audits the rubric with a second LLM pass, then scores each resume against it. Two showcase cases demonstrate both the system's value (13-pt MLE-vs-DataScientist spread on a TikTok recsys JD where the resumes genuinely differ along the JD's axes) and its honest limits (4-pt spread on a ByteDance ML-infra JD where neither resume actually has the depth signals the JD demands). **Key engineering finding:** migrating from a local 14B model (Ollama / Qwen 2.5) to a hosted 70B model (Groq / Llama 3.3) sped the pipeline up 15–30× and stabilized JSON outputs, but did not by itself improve rubric quality—the judgment-quality bottleneck lives in prompt design and pipeline structure, not in raw model capacity.

For reviewers / fastest path to the demo: open resumematch.pages.dev and click the "**Try with sample data**" button at the bottom of the form. This loads the TikTok showcase case (§5) and renders the full analysis in under 2 seconds—no backend cold-start, no uploads required. Custom uploads also work and trigger the live Groq pipeline (6–16 s; first request after a 15-minute idle may take 30–60 s for backend cold-start on Render's free tier).

1 Project Overview

ResumeMatch is a web application that helps job seekers quickly determine whether their resume matches a given job description (JD), and—when multiple resume versions are available—which version is the strongest fit.

The original motivation is the simple, day-to-day version of the problem: a job description is long, and a candidate just wants a fast structured answer to "does my resume fit this role, and where are the gaps?" The system extends naturally to a second use case: when the candidate maintains several resume versions (e.g., one for data science, one for ML engineering, one for research) and the JD title alone does not clearly indicate which framing the role actually wants, the system ranks all versions against the same JD and explains the result.

The system supports:

- Uploading multiple resume PDFs (with selection toggles to control which versions enter the analysis without removing files)
- Pasting a target job description
- Generating a structured JD summary (company, role, location, salary, key skills, hard requirements vs. nice-to-haves)

- Producing per-resume match scores across four dimensions (Skills, Experience, Education, Industry)
- Identifying strengths and gaps with explicit citations to both the resume and the JD
- On-demand improvement suggestions for each gap, including bullet rewrites grounded in existing resume content

The deployed system runs a fully cloud-hosted NLP pipeline (Groq-served Llama 3.3 70B on the backend, Cloudflare Pages serving a static Next.js frontend) and is available 24/7 at the URL above. An earlier iteration of the system ran the same pipeline locally on Ollama; the migration to a hosted backend is described in Section 3 and the engineering motivation in Section 9.

2 Problem Motivation

Resume–job matching is traditionally framed as an HR/recruiter problem—automated systems screen candidates on behalf of employers. ResumeMatch flips this perspective: it is a tool for the *job seeker*. The user owns and uploads their own resume(s); the system helps them choose and refine.

This framing matters because:

1. **JDs are long; the question is short.** Most job postings are 800–2,000 words mixing company branding, mission language, perks, legal disclaimers, and a small core of actual requirements. A candidate just wants to know whether their resume fits and what is missing. ResumeMatch produces that answer in seconds.
2. **Different resumes for different roles.** An ML engineer applying to a startup vs. a research lab vs. a finance firm should not submit identical resumes. Many candidates already curate multiple versions but have no systematic way to compare them against a specific opening—especially when the title is misleading (a "Machine Learning Engineer" posting whose actual requirements look more like a data scientist role, or vice versa).
3. **Privacy.** The pipeline only forwards resume text to a hosted LLM service at analysis time; it does not store user uploads, and the demo case is served from a cached response so casual visitors never trigger a network call to the model. This is meaningful for sensitive job-search activity.
4. **Honest, evidence-based feedback.** Recruiter-side tools optimize for the employer. A seeker-side tool can give blunt, evidence-grounded feedback ("this gap is real, here is the JD line that demands it, here is the resume line that misses it") instead of marketing the candidate.

3 System Architecture

3.1 Pipeline

```
User Browser (https://resumematch.pages.dev)
        |
        v
Next.js Frontend (Cloudflare Pages, static export)
        | upload PDFs + JD text (or load cached sample)
        v
FastAPI Backend (Render free tier, 24/7)
        |
        +--> PDF Parser (pdfplumber -> PyMuPDF fallback)
```

```
        |        extract resume text
        |
        +--> JD Parser (LLM, Llama 3.3 70B via Groq)
        |     - Extract structured JD summary (JSON)
        |     - Classify requirements (hard / nice-to-have / job duties)
        |     - Generate per-JD evaluation rubric (5-9 criteria)
        |
        +--> Rubric Self-Audit (LLM)
        |     Flag NICE_TO_HAVE_AS_HARD / JOB_DUTY_AS_REQUIREMENT /
        |     UNREASONABLE_THRESHOLD / NOT_IN_JD; remove or demote
        |
        +--> Per-Resume Matcher (LLM)
        |     For each rubric criterion: yes / partial / no
        |     + evidence excerpt + reasoning
        |
        +--> Scoring (deterministic, in Python)
        |     Map verdicts -> numeric score per dimension
        |     Aggregate weighted overall score
        |
        +--> Comparison & Insight (LLM)
                Identify Best Version, generate insight
        |
        v
Frontend renders:
  - JD summary card
  - Ranked result cards (collapsible)
  - Radar charts + dimension bars
  - Strengths / Gaps with cited excerpts
  - On-demand suggestions
```

3.2 Tech Stack

Layer	Technology	Reason
LLM	Llama 3.3 70B (Groq API)	Fast inference; reliable JSON output
Backend	FastAPI + Pydantic	Auto OpenAPI docs; type-safe schemas
Backend Host	Render (free tier)	Free, persistent, auto-deploy from GitHub
PDF	pdfplumber + PyMuPDF	Primary + fallback parser for robustness
Frontend	Next.js 15 + TypeScript	App Router, static export
UI	Tailwind v4 + shadcn/ui	Modern accessible component library
Visualization	Recharts	Radar charts for dimension scores
File Upload	react-dropzone	Drag-and-drop UX
HTTP	axios	Long timeout support
Frontend Host	Cloudflare Pages	Free, global CDN, custom domain

3.3 Hosted vs. Local LLM: A Considered Trade-off

An earlier iteration of this project used a local 14B model (Qwen2.5-14B Q4, ∼9 GB) via Ollama, motivated by:

- No API costs, repeatable demos
- Strict locality of resume text (a privacy argument)
- Educational value: demonstrating that meaningful NLP applications can be built without frontier-model API access

In practice, the local setup imposed two operational costs that proved unacceptable for a demo intended to be publicly reachable: (i) inference latency of 1–3 minutes per multi-resume analysis on M4 Max, and (ii) the backend was only reachable while the author's machine was online and an ngrok tunnel was active. Migrating the same pipeline to Groq's hosted Llama 3.3 70B reduced end-to-end latency to 6–16 seconds (a 15–30× speedup) and made the backend permanently reachable from any browser. The privacy argument is preserved by the cached-sample design (Section 8): the publicly visible demo case never hits the network at all, and live uploads are not persisted server-side.

The interesting finding from this migration is that the upgrade in model size (14B → 70B) and inference engine (Ollama CPU/GPU → Groq's custom inference hardware) materially improved *speed* and JSON-output *stability*, but did not by itself improve the *quality of judgment* the system produces. The judgment-quality bottleneck lives in the prompt design and the structure of the pipeline, not in raw model capacity. This is discussed further in Section 8.

4 NLP Methods and Course Connections

The system applies several NLP techniques covered in the course:

4.1 Structured information extraction

The JD parser asks the LLM to return a fixed JSON schema covering company, title, location, salary, work mode, education, hard requirements, nice-to-haves, and *job duties* (responsibilities described as future work, separated from hiring criteria). This is a form of **information extraction with constrained generation**, equivalent in spirit to template-filling but using a generative model with JSON-mode decoding.

4.2 Dynamic rubric generation

The most distinctive technical choice in this project is that the system does not use a fixed rubric. Instead, for each JD, the LLM generates a custom evaluation rubric of 5–9 criteria, each tagged with a target dimension (skills / experience / education / industry) and a weight. A rubric for a TikTok recommendation-systems internship will surface criteria like "production ML pipeline experience" and "ranking & retrieval familiarity"; a rubric for a quantitative research role will surface "in-depth research project on real-world data" and "hypothesis testing experience" instead.

The rubric generation prompt was iterated several times. Key refinements (Section 9 traces the iteration history):

- **Compound splitting:** Hard requirements written as conjunctions in the JD ("deep learning framework *and* familiarity with deep neural network architectures") must be split into atomic items so each can be evaluated separately.
- **Duty-to-experience proxy:** Technical-core responsibilities ("develop and maintain the model-as-a-service platform, including offline training/finetuning, online inference, model management, and resource orchestration") are transformed into past-experience proxy items ("has past experience fine-tuning large language models?", "has past experience deploying ML inference sys-

tems?"). Without this rule, the rubric trivially restates the qualifications section and misses the strongest signals the JD is providing.

- **Selective inclusion of nice-to-haves:** High-signal preferred qualifications (e.g., "experience in ModelOps," "LLM alignment research") enter the rubric at low weight (2–3) rather than being dropped, because they often discriminate among otherwise similar candidates.

Per-resume evaluation then becomes a structured judgment task: for each criterion, the LLM returns a `yes / partial / no` verdict, an evidence excerpt from the resume, and a brief reasoning. The final numeric scores are computed deterministically in Python from these verdicts (yes = 1.0, partial = 0.5, no = 0.0, weighted by criterion weight, mapped onto a 40–100 dimension score). This separation matters: the LLM only does what LLMs do well (read text, judge fit, cite evidence), and the scoring math is exact and reproducible.

4.3 Post-hoc Rubric Self-Audit

Even with a refined generation prompt, the rubric the LLM produces in a single pass is not always clean. Three failure modes appeared repeatedly across test JDs:

1. *Nice-to-haves treated as hard requirements.* The JD says "experience with Kubernetes is a plus," and the rubric scores it at weight 8.
2. *Unreasonable thresholds.* The JD is for an internship, but the rubric demands "first-author publications in NeurIPS / ICML / ICLR."
3. *Items not present in the JD.* The rubric introduces criteria the JD does not mention at all.

To address this, the pipeline includes a second LLM pass—a **self-audit**—that re-reads the generated rubric in the presence of the original JD and tags each item as `keep`, `demote_to_low_weight`, or `remove`, with one of four diagnostic labels:

Label	Action
`NICE_TO_HAVE_AS_HARD`	Demote weight to 2
`JOB_DUTY_AS_REQUIREMENT`	Remove (if abstract) or keep (if duty's technical core)
`UNREASONABLE_THRESHOLD`	Demote weight to 2
`NOT_IN_JD`	Remove

The audit is fail-open: if the audit call fails (e.g., transient API error), the original rubric is used unchanged so the user-facing analysis is never blocked.

A subtle design tension arises here. The audit is strict, and on early iterations it deleted legitimate duty-to-experience proxy items (Section 4.2) because their surface wording differs from the JD literal text. The fix was to tighten the audit prompt to recognize that a past-experience proxy for a JD-stated duty is legitimate—only abstract activities like "collaboration" or "communication" should be removed as `JOB_DUTY_AS_REQUIREMENT`. This interaction between the generation prompt and the audit prompt is documented further in Section 9.

4.4 LLM-as-judge for matching

Instead of relying solely on a similarity score from embeddings, the matching is performed by an LLM that reads both texts and produces structured, criterion-by-criterion assessments. This is the **LLM-as-judge** pattern. For this application, the chief advantage over a single similarity number is interpretability: each piece of the score is tied to a specific JD requirement and a specific resume passage.

4.5 Document-level evidence linking

For every strength and gap (and every rubric verdict), the LLM is required to provide:

- A `resume excerpt` (quote from the candidate's text), and
- A `jd excerpt` (quote from the JD requirement it matches or lacks).

This makes the output auditable—the user can verify each claim by tracing it back to the source. This connects to the course theme of evidence-based text analysis.

4.6 Generation with hard constraints

The improvement-suggestion feature must rewrite resume bullets without inventing experience. The system prompt enforces this with explicit verification steps. This is a **controlled generation** problem; LLM-only solutions are imperfect, but the prompt structure (rules + verification checklist) reduces fabrication substantially. Where rewrites cannot be safely produced, the system returns higher-level advice instead.

5 Showcase Examples

5.1 Case 1: TikTok ML Engineer Intern (Recommendation)

To illustrate the system end-to-end, I evaluated two versions of my own resume against TikTok's Machine Learning Engineer Intern (Recommendation) posting. The two resumes describe the same person and contain overlapping content, but are framed differently:

- **DataScientist version:** emphasizes A/B testing, hypothesis testing, KPI analysis, and luxury brand marketing experience; describes the same Vitongue ML internship at a high level focused on cross-team coordination.
- **MLE version:** explicitly surfaces a "Production ML" skill block (real-time inference, sub-800ms latency, deployment pipelines), an "Applied ML: Ranking & Retrieval" skill block, and describes the same Vitongue project in terms of RetinaFace + FaceNet face recognition, Whisper-based semantic indexing, and Milvus vector retrieval.

The system extracted six hard requirements and four nice-to-haves from the JD, and—after self-audit—produced a seven-item evaluation rubric covering: Master's degree, deep learning framework proficiency, ranking/retrieval/recommendation experience, production ML pipeline experience, content/LLM/robustness ML experience, hypothesis-testing experience, and first-author publications at top AI conferences (low weight).

Resume version	Skills	Experience	Education	Industry	Overall
MLE version	100	96	70	75	**88 (Best)**
DataScientist version	100	57	70	75	75

The 13-point gap was driven primarily by the production-ML and ranking/retrieval rubric items, which the MLE version satisfied with direct evidence ("Implemented semantic indexing with Whisper (large-v3-turbo) and Milvus vector database...", "<800ms inference latency") while the DataScientist version did not surface them. The first-author publications criterion is correctly weighted lightly because it is a preferred (not required) qualification, and both resumes fail it equally.

5.2 Case 2: ByteDance ML Engineering Intern (a less favorable case)

A second test case revealed an important limitation. The same two resumes were evaluated against ByteDance's Applied Machine Learning Enterprise intern posting, which is also an "ML engineering intern" role but emphasizes large-scale distributed training/inference systems, GPU/RDMA/Storage infrastructure, and ModelOps—not recommendation systems.

After prompt refinement (Section 9), the rubric grew to nine items covering CS-degree, deep learning frameworks, programming languages, large-scale distributed systems, LLM fine-tuning, ModelOps/lifecycle, compute resource orchestration, inference deployment, and top-tier publications (low weight)—a strong coverage of the JD's depth signals. The two resumes scored, however, only four points apart (MLE 74, DataScientist 70).

This is not a failure of the rubric. The MLE-framed resume's strongest differentiators (Milvus vector database, sub-800ms latency, GPU optimization, production deployment) are partial matches for "large-scale distributed inference systems" but only weak matches for "has fine-tuned LLMs," "ModelOps workflow," or "compute resource orchestration"—all of which the resume genuinely does not have, because the candidate has not done that specific kind of work. The DataScientist version trivially fails the same items. The verdict "these two resumes are similarly under-qualified for this specific role" is, in this case, the correct answer.

The case is instructive nonetheless: it shows that the system's discriminative power is upper-bounded by the actual content difference between resume versions, and that a job seeker who repeatedly receives flat comparisons on a class of JD is being given honest signal that their resume framing is not the gap—the underlying experience is.

6 Key Design Decisions

6.1 Single-user, single-page UX

No login, no history, no database. The app is intentionally stateless—paste, analyze, leave. This simplifies the system and reflects the real workflow: a job seeker analyzing one application at a time.

6.2 Adaptive single-page layout

The same page handles 1 resume vs. N resumes. With one resume, the result card is fully expanded. With multiple, only the best version expands by default; others collapse to a one-line header showing score and filename. This avoids forcing the user into different screens for what is conceptually the same task.

6.3 Selection toggles instead of delete-and-reupload

Each uploaded resume has a checkbox. The user can toggle which versions enter the analysis without removing files. This supports the realistic workflow where a user has a small library of resume variants and wants to compare different subsets against different JDs over time.

6.4 Filename-based identity

Comparison output references resumes by filename (e.g., `ZhuweiXu_MLE_Resume.pdf`) rather than internal IDs (`resume_1`). Users name their files meaningfully, and the system should respect that.

6.5 Strict gap definition

After observing LLM tendency to over-flag soft preferences as gaps, the prompt now distinguishes:

- Hard requirement clearly missing → *gap*
- Nice-to-have, post-hire activity, or soft cultural trait → *not* a gap (or weighted very lightly in the rubric)

This required iterating on the prompt across several test cases, including adversarial JDs with mixed required/preferred language, and is reinforced by the rubric self-audit (Section 4.3).

6.6 Deterministic scoring on top of LLM judgments

As described in Section 4.2, the LLM returns categorical verdicts and the numeric aggregation happens in Python. This produces stable, reproducible scores from a given set of verdicts and prevents the LLM from drifting on the math (e.g., assigning 87 to one resume and 86 to a clearly weaker one).

6.7 Cached sample case for public demo reliability

The public landing page exposes a "Try with sample data" button that loads a hand-curated, pre-computed analysis of the TikTok JD (Case 1). This response is served from a static JSON file shipped with the frontend and does not hit the backend at all. The motivation is reliability: the cached path is fast (under 2 s) and immune to backend cold starts (Render's free tier sleeps after 15 minutes of inactivity, adding 30–60 s to the first request after sleep). Live uploads of user-provided resumes always trigger the full backend pipeline.

7 Demo Walkthrough

The deployed app at `https://resumematch.pages.dev` is publicly accessible. The intended walkthrough:

1. **Click "Try with sample data"** — Loads two pre-included resume versions (DataScientist + MLE) and the TikTok JD described in Case 1. The result renders in under two seconds because it is served from the cached response.
2. **Inspect the resume list** — Both versions appear with selection checkboxes; deselect a version to exclude it from analysis.
3. **Click "Analyze Match"** — The system displays the JD summary card (location, salary, key skills, requirements breakdown) followed by ranked result cards.
4. **Inspect the Best Version card (MLE, score 88)** — Radar chart, dimension scores, and three collapsible sections: Rubric Evaluation, Strengths, Gaps. Each strength and gap shows the resume excerpt alongside the JD excerpt it relates to.
5. **Expand a gap** — Click to receive an improvement suggestion (and a concrete bullet rewrite where applicable).
6. **Compare to the other version (DataScientist, score 72)** — Same structure, with different rubric verdicts and different gaps surfaced.

For custom inputs (uploading the user's own resume and JD), the same flow runs against the live Groq-served backend. First request after a period of inactivity will incur a 30–60 s backend cold start; subsequent requests complete in 6–16 s.

8 Limitations and Future Work

8.1 Honest assessment of what the system does and does not do

This project crossed an interesting boundary during development: the parts of the system that look engineered (deployment, latency, cached fallback, type-safe schemas, deterministic scoring) reached production quality, while the part of the system that does the actual judgment work (the LLM's interpretation of a JD into a rubric) remained *good but JD-dependent*.

Three concrete limitations to be candid about:

Rubric quality is JD-style sensitive. The TikTok recommendation-systems JD (Case 1) produces a strongly differentiating rubric, and the MLE-framed resume cleanly beats the DataScientist-framed resume by 13 points. The ByteDance ML infra JD (Case 2) produces a comprehensive nine-item rubric covering depth signals, but the two resumes land four points apart because the depth signals the JD asks about are signals neither resume actually has. The rubric is doing the right thing in both cases; the discriminative power simply depends on whether the candidate's resume variants differ along the dimensions the JD actually cares about.

Model size does not fix judgment quality. Migrating from a local 14B model to a hosted 70B model improved JSON-output stability and inference speed by an order of magnitude, but did not by itself produce more discriminating rubrics. The bottleneck on rubric quality lives in two places that a larger model does not address: (i) the prompt must encode *how* to read a JD (split compounds, extract duties, infer technical depth), which is a design problem rather than a capacity problem; and (ii) when a JD's surface text shares vocabulary with many candidate profiles, no amount of model capacity can manufacture signal that is not present in the input. This finding is consistent with the broader observation that LLM applications often hit ceilings determined by prompt design and input quality long before they hit ceilings determined by raw model capability.

Two-pass design has its own pathology. The generation prompt and the audit prompt interact. Early iterations of the audit deleted legitimate duty-to-experience proxy items (Section 4.3) because their surface wording differed from the JD's literal text. Tightening the audit fixed this case but raises the broader question of whether stacking LLM passes is the right approach versus, e.g., a single more carefully structured generation pass with explicit chain-of-thought, an ensemble of multiple generations voted into a consensus rubric, or a small fine-tuned model trained on labeled `(JD, rubric)` pairs. The current two-pass design was chosen as the simplest improvement that demonstrably reduced visible rubric errors, and is acknowledged as a partial fix.

8.2 Other limitations

- **Cold start:** The Render free tier sleeps after 15 minutes of inactivity, adding 30–60 s to the first request after sleep. The cached sample-data path avoids this entirely, but custom uploads after a period of quiet experience the delay.
- **Rate limits:** The Groq free-tier daily token budget supports roughly 6–10 full multi-resume analyses per day. A small public demo is fine; sustained external use would require either a paid Groq tier or a request queue with friendly degradation.
- **No persistence:** Past analyses are not saved. The user loses results on refresh.
- **English-only:** The prompts and UI are English-only. Non-English resumes/JDs work but quality varies.

- **No anti-hallucination guarantee:** Despite prompt engineering, the LLM occasionally suggests rewrites that subtly stretch the truth. The user must still verify suggestions.

8.3 Future directions

The most important next step is **human-in-the-loop rubric review**: after the LLM proposes a rubric and self-audits it, the user is shown the rubric and can delete, edit, reweight, or add items before scoring runs. This pattern (proposal + user accept/reject) is how Grammarly, Copilot, and similar AI products handle the fact that the underlying model is good-but-not-perfect. It moves rubric quality from "trust the LLM to be right every time" to "LLM proposes, user validates," which is both more honest and more useful.

Other directions:

- **Rubric ensemble:** generate the rubric three times with different prompt variants, keep items appearing in at least two runs. This is a cheap way to harden against single-run instability.
- **Streaming results:** show the JD summary first, then each resume as it completes, instead of waiting for the full batch.
- **Resume version history** stored in browser `localStorage` (no backend persistence required).
- **Bilingual support** with parallel prompts for Chinese resumes/JDs.
- **Company research integration:** click company name → fetch and summarize publicly available info.
- **Embedding-based panel:** for users curious about semantic-similarity baselines, expose a TF-IDF / sentence-embedding score alongside the rubric-based score to highlight where the two methods disagree.
- **Fine-tuning on labeled rubric data**: with 200–500 labeled `(JD, ground-truth-rubric)` pairs, a small fine-tuned model could likely outperform the current two-pass prompt design, at the cost of having to collect the data.

9 Engineering Iteration Notes

This section documents the iteration history of the matching pipeline. Each iteration was driven by a specific failure mode observed on a real test case.

Iteration 1: Local Ollama baseline. The first working version of the system used Ollama serving Qwen 2.5 14B locally on the author's machine, exposed publicly via an ngrok tunnel during demos. The pipeline produced reasonable rubrics on the TikTok JD case, but suffered from two operational problems: per-analysis latency of 1–3 minutes and the obvious requirement that the author's laptop be online for the demo to work.

Iteration 2: Migration to hosted inference. The same pipeline (no prompt changes) was migrated to Groq's hosted Llama 3.3 70B. End-to-end latency dropped to 6–16 s, and JSON-output stability improved noticeably (the local 14B model occasionally produced malformed JSON that required regex repair; the 70B model essentially never does). Backend was redeployed to Render's free tier as a FastAPI service, with the frontend continuing to be served from Cloudflare Pages. No frontend logic changed except the API base URL.

Iteration 3: Refining JD classification. On adversarial JDs with mixed required/preferred language ("familiarity with Kubernetes is a plus," "Redis is preferred"), the old prompt classified these as hard requirements. The fix was to replace the "when in doubt, prefer hard" instruction with an explicit screening-test heuristic ("would a candidate without this be filtered at resume screening?"), and to distinguish whole-item qualification (entire bullet is preferred) from sub-clause qualification ("(AWS preferred)" inside an otherwise required bullet).

Iteration 4: Adding post-hoc rubric audit. Even with refined classification, the rubric still occasionally included unreasonable thresholds ("first-author NeurIPS publication" for an internship), or nice-to-haves dressed up as hard requirements. The audit pass (Section 4.3) was added at this point. On a synthetic adversarial test rubric, the audit correctly demoted three nice-to-haves to weight 2 and removed two items entirely (one job duty re-encoded as past experience, one not in the JD at all).

Iteration 5: Compound splitting and duty-to-experience proxy. On the ByteDance JD, the rubric initially only restated the Qualifications section and missed the entire Responsibilities section, where the JD's strongest signals lived ("offline training/finetuning," "online inference," "model management and resource orchestration," "GPU resource management"). Two prompt changes addressed this:

- *Compound splitting:* hard requirements written as conjunctions in the JD must produce one rubric item per conjunct.
- *Duty-to-experience proxy:* technical-core duties in the Responsibilities section must be transformed into past-experience proxy items.

Iteration 6: Audit tightening to match generation changes. After iteration 5, the audit aggressively deleted the new duty-to-experience proxy items, marking them `NOT_IN_JD` because their surface wording ("has past experience deploying ML inference services?") differed from the JD's literal text ("develop and maintain ... online inference ..."). The audit prompt was tightened to recognize past-experience proxies for JD-stated duties as legitimate. After this change, the ByteDance rubric grew to nine items with coverage across all four depth signals (LLM fine-tuning, inference deployment, ModelOps/lifecycle, large-scale/distributed systems).

Where iteration stopped. After iteration 6, the rubric on both Case 1 and Case 2 was substantively complete. Further prompt iteration would likely produce small marginal gains on specific JDs at the cost of regressions on others; this is the classic prompt-engineering plateau where the right next investment is structural (human review, ensemble, fine-tuning) rather than incremental prompt edits.

10 Replication

10.1 Local setup

```
# 1. Get a Groq API key
# Sign up at https://console.groq.com, create an API key,
# and export it: export GROQ_API_KEY=gsk_...

# 2. Backend
cd backend
conda create -n resume-match python=3.11 -y
conda activate resume-match
pip install -r requirements.txt
```

```
uvicorn app.main:app --reload --port 8000

# 3. Frontend (separate terminal)
cd frontend
npm install
npm run dev
```

Open http://localhost:3000.

10.2 Cloud deployment (the production setup)

Backend is deployed to Render via a `render.yaml` Blueprint at the repository root, which pins Python 3.11.10 and exposes `GROQ_API_KEY` as a manually-set environment variable. The frontend is built and served by Cloudflare Pages, configured to deploy on every push to `main`. The frontend's API base URL is hard-coded to the Render service URL as a production fallback, so the build is not dependent on a Cloudflare-side environment variable.

10.3 Repository structure

```
ResumeMatch/
|-- backend/
| `-- app/
| |-- main.py # FastAPI entry point
| |-- matcher.py # Core matching logic, prompts, scoring,
| | # and rubric self-audit
| |-- llm_client.py # Groq client wrapper
| |-- pdf_parser.py # PDF text extraction
| `-- schemas.py # Pydantic models
|-- frontend/
| |-- app/ # Next.js App Router pages
| |-- components/
| | `-- resume-match/ # Domain-specific components
| |-- lib/ # API client + utilities
| `-- public/samples/ # Sample resumes + JD + cached response
|-- render.yaml # Render Blueprint for backend deployment
`-- README.md
```

11 Acknowledgments

This project was developed using Claude (Anthropic) as a pair-programming assistant for code generation, debugging, and prompt iteration, consistent with the course's encouragement of AI-agent-assisted development.